

Can Computers Think? (CSC 106)

Practice for the Practical Exam

Format

- 1 hour
- 4 tasks
- Included relevant python documentation
- code in IDLE
- submit on Gradescope
- some written answer questions that will be collected

Topics

- Values and Types
- Operations
- Strings
- ASCII
- Input
- Printing / Returning
- Loops
- Conditionals
- Lists
- Dictionaries

Practice Problem 1: Write a function that adds up multiples of 9

- name your function `sum_nines` and your file `sum_nines.py`
- the function should
 - + ask the user for input
 - + let the user know how to end the program
 - + check whether the input is a number that is a multiple of 9
 - + add multiples of 9 to a running total
 - + return the total
- start with the most simple working version, then add complexities
- don't forget to comment you code
- between defining and testing your function put the following comment
 - `### BEGIN TESTING`
- this activity should take around 15 mins

Practice Problem 2: Write a function that makes lists alliterative

- name your function `al_lists` and your file `char_lists.py`
- the function should:
 - + take a lists of words and a character as parameters
 - + return a new list only containing words that start with the character
 - + be robust to edge cases (empty lists, maybe lists without strings)
 - + optionally alter the function to take an ASCII value instead of a char
- start with the most simple working version, then add complexities
- don't forget to comment you code
- between defining and testing your function put the following comment
 - `### BEGIN TESTING`
- this activity should take around 15 mins

Practice Problem 3: Change the function above to take an ascii value instead of a character

- name your function `ascii_lists` and your file `ascii_char_lists.py`
- the function should:
 - + take a lists of words and an integer as parameters
 - + return a new list only containing words that start with the ascii value character
- start with the most simple working version, then add complexities
- don't forget to comment you code
- between defining and testing your function put the following comment
 - `### BEGIN TESTING`
- this activity should take around 5 mins

Practice Problem 4: Write a program that helps the user study

- name your function `study_tool` and your file `study_guide.py`
- the program should:
 - + initiate a dictionary of CSC106 topics and in-class activities
 - + name activities based on their week
 - + decide what info will be keys and what will be values
 - + decide if you need to use strings, lists, nested lists, nested dictionaries, etc.
 - + include a function that looks up activities based on a topic
 - + optionally include information on whether you completed each activity
 - + optionally include descriptions of each activity
- start with the most simple working version, then add complexities
- don't forget to comment you code
- between defining and testing your function put the following comment
 - `### BEGIN TESTING`
- this activity should take around 20 mins

The following is a list of activities and topics you can use

- * W2-2 : for loops, input
- * W2-3 : memory diagrams and tracing
- * W3-3 : math facts
- * W3-4 : conditionals
- * W4-1 : multi-branch conditionals (Reeborg)
- * W4-2 : conditionals
- * W5-1 : while loops
- * W5-2 : ASCII
- * W6-1 : lists and strings
- * W6-2 : Caesar cipher, strings
- * W6-3 : lists, numbers and strings
- * W7-1 : lists of numbers
- * W7-2 : nested lists
- * W7-3 : dictionaries and lists
- * W8-1 : reading/writing files
- * W8-2 : dictionaries for translation
- * W8-3 : author identification

Name or initials:

BEFORE starting your code, explain your program design (what decisions did you make based on the above prompt, which built-in functions will you need based on the below documentation, etc.)

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the page.

Documentation

While loops:

- **Syntax:**
while <condition>:
 <statements>
 <following statements>
- **Semantics:** Execute the indented <statements> until the <condition> is False....then move on to the <following statements>.

Strings:

- <string name> + <string name> : concatenation
- <string name> * <int> : duplication/replication
- in : checking containment:
 Example use: if <character> in <string name>:
- len(<string name>) : getting length
- <string name>[<i>] : indexing
- <string name>[<start>:<stop>] : slicing
- <string name>.strip("<something>")
- <list of strings> = "a string value".split()
 Semantics: creates a list of strings by dividing up the specified string using white space

Lists:

- <list name> + <list name> : concatenation
- in : checking containment:
 Example use: if <element> in <list name>:
- len(<list name>) : getting length
- <list name>[] : indexing
- <list name>.append() : used to add a single element to the end of a list. This element can be any data type, a number, a string, another list, or even an object.
- <list name>.extend() : used to add all elements from an iterable (e.g., a list, tuple, or set) to the end of the current list. Unlike append(), which adds an entire iterable as a single element, extend() adds each element from the iterable to the list.
- <list name>[start:stop] : slicing

ASCII:

- chr(<integer>): get the character associated with the integer.
- ord(<character>): get the ASCII value of the character

Dictionaries:

- Access value by key: <dict name>[<key>]
- Add or update value: <dict name>[<key>] = <value>
- <key> in <dict name>: Returns True if <key> is in the dictionary, else False. Example use:
 if <key> in <dict name>:
- Get all keys: <dict name>.keys()
- Get all values: <dict name>.values()
- Get all key-value pairs: <dict name>.items()
- Get length: len(<dict name>)

Name or initials: